

UCS 2312 Data Structures Lab
Assignment 7: Implementation of AVL Tree

QUESTION 1 : Design an ADT for the AVL Tree data structure with the following functions. Each node consists of a character data, address of left, right and parent nodes [CO1, K3]

a. insertAVL(t, data) – insert data into BST

ALGORITHM:

INPUT:

i) struct tree *

ii) int

OUTPUT: struct tree *

```
1. if (t==NULL)
    {
        t=(struct tree *)malloc(sizeof(struct tree));
        t->data = x;
        t->height = 0;
        t->left=t->right=NULL;
    }
2. else if (x<t->data)
    {
        t->left = insert(t->left,x);
        if (height(t->left)-height(t->right)==2)
        {
            if (x<t->left->data)
                t=sll(t);
            else
                t=drl(t);
        }
    }
3. else if (x>t->data)
    {
        t->right=insert(t->right,x);
        if (height(t->right)-height(t->left)==2)
        {
            if (x>t->right->data)
                t=srr(t);
            else
                t=dlr(t);
        }
    }
4. t->height = max(height(t->left),height(t->right))+1;
```

5. return t;

b. hierarchical(t) – display the tree in hierarchical fashion

ALGORITHM:

INPUT:

i)struct tree *

OUTPUT: None

```
1. if (t!=NULL)
{
    print(t->data,x);
    if (t->left!=NULL)
        levelorder(t->left,x+1);
    if (t->right!=NULL)
        levelorder(t->right,x+1);
}
```

c. findParent(t, key) – will return the parent of the given data

ALGORITHM:

INPUT:

i)struct tree *

OUTPUT: int

```
1. if (t!=NULL)
{
    if (t->left->data ==k || t->right->data==k)
        return t->data;
    else if (k<t->data)
        findparent(t->left,k);
    else if (k>t->data)
        findparent(t->right,k);
}
```

CODE:
Treadt.h

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
struct tree
{
    int data,height;
    struct tree *left,*right;
};

int max(int a,int b)
{
    if (a>b)
        return a;
    else
        return b;
}

int height(struct tree *t)
{
    if (t==NULL)
        return -1;
    else
        return t->height;
}

struct tree *sll(struct tree *k2)
{
    struct tree *k1;
    k1=k2->left;
    k2->left=k1->right;
    k1->right=k2;
    k2->height= max(height(k2->left),height(k2->right))+1;
    k1->height = max(height(k1->left),height(k1->right))+1;
    return k1;
}

struct tree *srr(struct tree *k1)
{
    struct tree *k2;
    k2=k1->right;
    k1->right = k2->left;
```

```

    k2->left=k1;
    k1->height = max(height(k1->left),height(k1->right))+1;
    k2->height= max(height(k2->left),height(k2->right))+1;
    return k2;
}

struct tree *dlr(struct tree *k1) //left subtree right child
{
    k1->right = sll(k1->right);
    return srr(k1);
}

struct tree *drl(struct tree *k1) //right subtree of left child
{
    k1->left = srr(k1->left);
    return sll(k1);
}

struct tree *insert(struct tree *t,int x)
{
    if (t==NULL)
    {
        t=(struct tree *)malloc(sizeof(struct tree));
        t->data = x;
        t->height = 0;
        t->left=t->right=NULL;
    }
    else if (x<t->data)
    {
        t->left = insert(t->left,x);
        if (height(t->left)-height(t->right)==2)
        {
            if (x<t->left->data)
                t=sll(t);
            else
                t=drl(t);
        }
    }
    else if (x>t->data)
    {
        t->right=insert(t->right,x);
        if (height(t->right)-height(t->left)==2)
        {
            if (x>t->right->data)

```

```

        t=srr(t);
    else
        t=dlr(t);
    }
}
t->height = max(height(t->left),height(t->right))+1;
return t;
}

void inorder(struct tree *t)
{
    if (t->left!=NULL)
        inorder(t->left);
    printf("%d ",t->data);
    if (t->right!=NULL)
        inorder(t->right);
}

void print(int e,int x)
{
    for (int i =0;i<x;i++)
    {
        printf("\t");
    }
    printf("%d\n",e);
}

void levelorder(struct tree *t,int x)
{
    if (t!=NULL)
    {
        print(t->data,x);
        if (t->left!=NULL)
            levelorder(t->left,x+1);
        if (t->right!=NULL)
            levelorder(t->right,x+1);
    }
}

int findparent(struct tree *t,int k)
{
    if (t!=NULL)
    {
        if (t->left->data ==k || t->right->data==k)

```

```

        return t->data;
    else if (k<t->data)
        findparent(t->left,k);
    else if (k>t->data)
        findparent(t->right,k);
    }
}

```

Tree.c

```

#include <stdio.h>
#include <stdlib.h>
#include "treeadt.h"
void main()
{
    struct tree *t;
    t=NULL;
    printf("enter the number of nodes:");
    int n;
    scanf("%d",&n);
    for (int i = 0;i<n;i++)
    {
        printf("enter the value :");
        int a;
        scanf("%d",&a);
        t=insert(t,a);
    }
    printf("--INORDER---\n");
    inorder(t);
    printf("\n---LEVEL ORDER---\n");
    levelorder(t,0);
    printf("enter the key to find the parent : ");
    int k;
    scanf("%d",&k);
    printf("%d ",findparent(t,k));
}

```

OUTPUT:

```
C:\Users\User\OneDrive\SEMS\DATA STRUCTURES\LAB
enter the number of nodes:5
enter the value :10
enter the value :3
enter the value :188
enter the value :45
enter the value :34
--INORDER--
3 10 34 45 188
---LEVEL ORDER---
10
    3
    45
        34
        188
enter the key to find the parent : 34
45
```

```
enter the number of nodes:5
enter the value :1
enter the value :2
enter the value :3
enter the value :4
enter the value :5
--INORDER--
1 2 3 4 5
---LEVEL ORDER---
2
    1
    4
        3
        5
enter the key to find the parent : 1
2
```

QUESTION 2 : Demonstrate the AVL ADT with the insertion of the following character data one at a time.

H, I, J, B, A, E, C, F, D, G, K, L

CODE:

Chartreadt.h

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
struct tree
{
    char data;
    int height;
    struct tree *left,*right;
};

int max(char a,char b)
{
    if ((int)a>(int)b)
        return a;
    else
        return b;
}

int height(struct tree *t)
{
    if (t==NULL)
        return -1;
    else
        return t->height;
}

struct tree *sll(struct tree *k2)
{
    struct tree *k1;
    k1=k2->left;
    k2->left=k1->right;
    k1->right=k2;
    k2->height= max(height(k2->left),height(k2->right))+1;
    k1->height = max(height(k1->left),height(k1->right))+1;
    return k1;
}
```

```

struct tree *srr(struct tree *k1)
{
    struct tree *k2;
    k2=k1->right;
    k1->right = k2->left;
    k2->left=k1;
    k1->height = max(height(k1->left),height(k1->right))+1;
    k2->height= max(height(k2->left),height(k2->right))+1;
    return k2;
}

struct tree *dlr(struct tree *k1) //left subtree right child
{
    k1->right = sll(k1->right);
    return srr(k1);
}

struct tree *drl(struct tree *k1) //right subtree of left child
{
    k1->left = srr(k1->left);
    return sll(k1);
}

struct tree *insert(struct tree *t,char x)
{
    if (t==NULL)
    {
        t=(struct tree *)malloc(sizeof(struct tree));
        t->data = x;
        t->height = 0;
        t->left=t->right=NULL;
    }
    else if ((int)x<(int)t->data)
    {
        t->left = insert(t->left,x);
        if (height(t->left)-height(t->right)==2)
        {
            if ((int)x<(int)t->left->data)
                t=sll(t);
            else
                t=drl(t);
        }
    }
    else if ((int)x>(int)t->data)

```

```

    {
        t->right=insert(t->right,x);
        if (height(t->right)-height(t->left)==2)
        {
            if ((int)x>(int)t->right->data)
                t=srr(t);
            else
                t=dlr(t);
        }
    }
    t->height = max(height(t->left),height(t->right))+1;
    return t;
}

void inorder(struct tree *t)
{
    if (t->left!=NULL)
        inorder(t->left);
    printf("%c ",t->data);
    if (t->right!=NULL)
        inorder(t->right);
}

void print(int e,int x)
{
    for (int i =0;i<x;i++)
    {
        printf("\t");
    }
    printf("%c\n",e);
}

void levelorder(struct tree *t,int x)
{
    if (t!=NULL)
    {
        print(t->data,x);
        if (t->left!=NULL)
            levelorder(t->left,x+1);
        if (t->right!=NULL)
            levelorder(t->right,x+1);
    }
}

```

```

int findparent(struct tree *t,int k)
{
    if (t!=NULL)
    {
        if (t->left->data ==k || t->right->data==k)
            return t->data;
        else if (k<t->data)
            findparent(t->left,k);
        else if (k>t->data)
            findparent(t->right,k);
    }
}

```

Chartree.c

```

#include <stdio.h>
#include <stdlib.h>
#include "chartreeadt.h"

void main()
{
    struct tree *t;
    t=NULL;
    printf("enter the number of nodes:");
    int n;
    scanf("%d",&n);
    for (int i = 0;i<n;i++)
    {
        printf("enter the value :");
        char ch[1];
        scanf("%s",ch);
        t=insert(t,ch[0]);
    }
    printf("--INORDER---\n");
    inorder(t);
    printf("\n---LEVEL ORDER---\n");
    levelorder(t,0);
    printf("enter the key to find the parent : ");
    char k[1];
    scanf("%s",k);
    printf("%c ",findparent(t,k[0]));
}

```

OUTPUT:

```
enter the number of nodes:12
enter the value :H
enter the value :I
enter the value :J
enter the value :B
enter the value :A
enter the value :E
enter the value :C
enter the value :F
enter the value :D
enter the value :G
enter the value :K
enter the value :L
--INORDER---
A B C D E F G H I J K L
---LEVEL ORDER---
E
      C
        B
          A
            D
              H
                F
                  G
                    J
                      I
                        K
                          L
enter the key to find the parent : I
J
```

```
enter the number of nodes:5
enter the value :a
enter the value :b
enter the value :c
enter the value :d
enter the value :e
--INORDER---
a b c d e
---LEVEL ORDER---
b
      a
        d
          c
            e
enter the key to find the parent : a
b
```